



Universidad
Indoamérica

UNITAT 4 · FONAMENTS DE SISTEMES OPERATIUS

Processos i memòria

Universidad Tecnológica Indoamérica

Laboratori TD3

El procés: un programa en execució i el seu context

- Context: memòria, registres, estat dels descriptors...
- El SO executa molts processos alhora (unitat 1, A1.4)
- Cadascun té el seu propi **espai d'adreces privat**: aïllament, seguretat i l'abstracció de la memòria virtual
- Amaga el nucli i els altres processos

Adreçament virtual

- Una adreça d' n bits permet 2^n adreces
- Espai d'adreces = les adreces + l'estat de cada cel·la
- Cada procés només veu **el seu**

A x86-64 avui

Punters de 64 bits, però només se'n fan servir **48**: 256 TiB d'espai virtual, la meitat baixa per al procés i l'alta per al nucli. Amb paginació de 5 nivells, 57 bits (128 PiB).

Les zones d'un procés: pila, heap, dades, codi

Sortida real

```
$ ./zones
zone          address      perm  region
code   (main)    0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)      0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)        0x7fff6c1e68b4  rw-p  [stack]
```

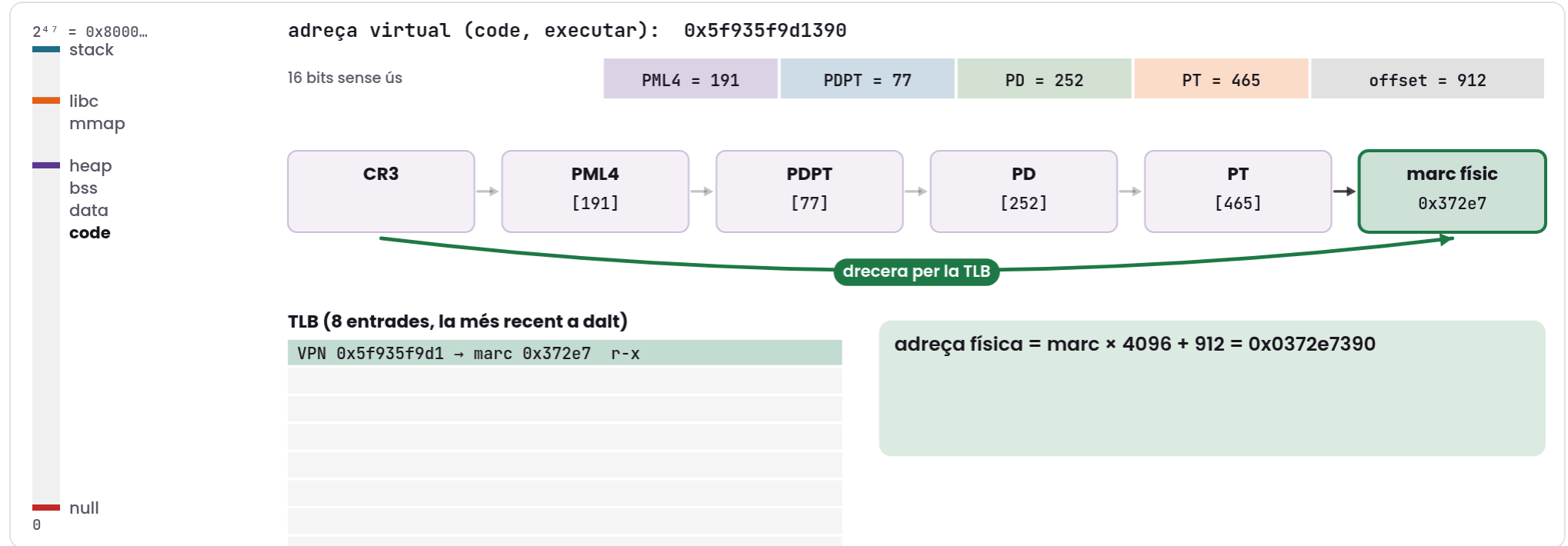
Pila: variables locals · **heap:** `malloc` petits · **mmap:** `malloc` grans i biblioteques · **data/bss:** globals i `static` · **codi** (text): les instruccions, només de lectura i executables.

Del virtual al físic: la paginació

- La CPU fa la traducció: la **MMU**, amb la memòria cau **TLB**
- La memòria es gestiona en **pàgines**: 4 KiB a x86-64 (i pàgines grans de 2 MiB o 1 GiB); a ARM també 16 o 64 KiB
- Cada pàgina virtual apunta a un **marc** físic, a través d'una taula (a x86-64, 4 nivells)
- L'adreça es divideix en camps: índexs de taula i desplaçament

Correcció a l'original: deia «blocs de 2 KB (4 KB o 2 MB a x86)». A x86 la pàgina base és de 4 KiB; 2 KB no és una mida de pàgina d'x86.

Traduir adreces reals



Assignació mandrosa: la pàgina existeix quan es toca

Sortida real

```
$ ./lazy 1024
start                                VmRSS =    1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =    1952 KiB    page faults so far = 8
after touching half of the pages     VmRSS =   526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each) page faults: 131073    1968 ns per fault
after free                            VmRSS =    2012 KiB    page faults so far = 131081
```

- Molt més eficient que assignar-ho tot per endavant
- I si no queden marcs lliures? Una pàgina poc usada va al disc (intercanvi) o es comprimeix a la RAM (zswap, zram): substitució de pàgines →

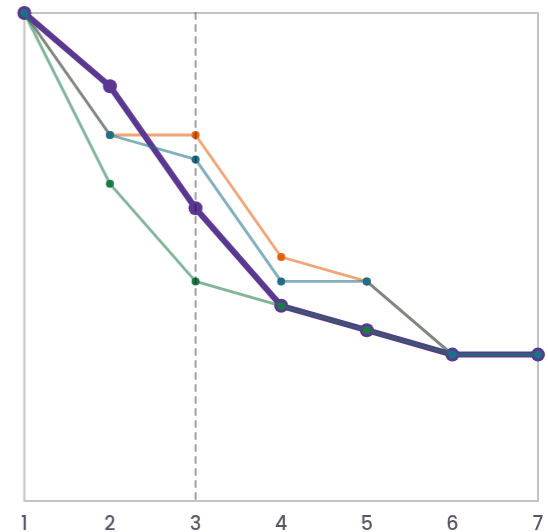
Sense marcs lliures: quina pàgina se'n va?

LRU · 3 marcs

ref.	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
marc 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
marc 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
marc 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
	F	F	F	F		F		F	F	F	F			F		F		F		
				-7		-1		-2	-3	-0	-4			-0		-3		-2		

F = fallada · -x = víctima

fallades segons els marcs



FIFO: 15

LRU: 12

Optim (OPT): 9

La interfície de memòria: `brk`, `malloc`, `free`

C

```
int brk(void *end);           /* end of the heap */
void *sbrk(intptr_t delta);
void *malloc(size_t size);    /* NULL on failure */
void *calloc(size_t n, size_t size);
void *realloc(void *p, size_t size);
void free(void *p);           /* free(NULL): no-op */
```

No facis servir `brk` / `sbrk` directament: fes servir `malloc`.

- glibc: blocs petits al heap (`brk`); a partir de 128 KiB, un `mmap` propi (visible a `zones`)
- Linux és optimista (*overcommit*): `malloc` gairebé mai no retorna NULL; si s'acaba la memòria, l'**OOM killer** acaba processos
- Doble `free` o `free` d'una cosa no assignada: comportament indefinit
- Depuració: GDB, Valgrind i avui `-fsanitize=address` (ASan)

El procés per al SO: el PCB

Estàtic

PID, PPID, propietari, drets d'accés

Dinàmic

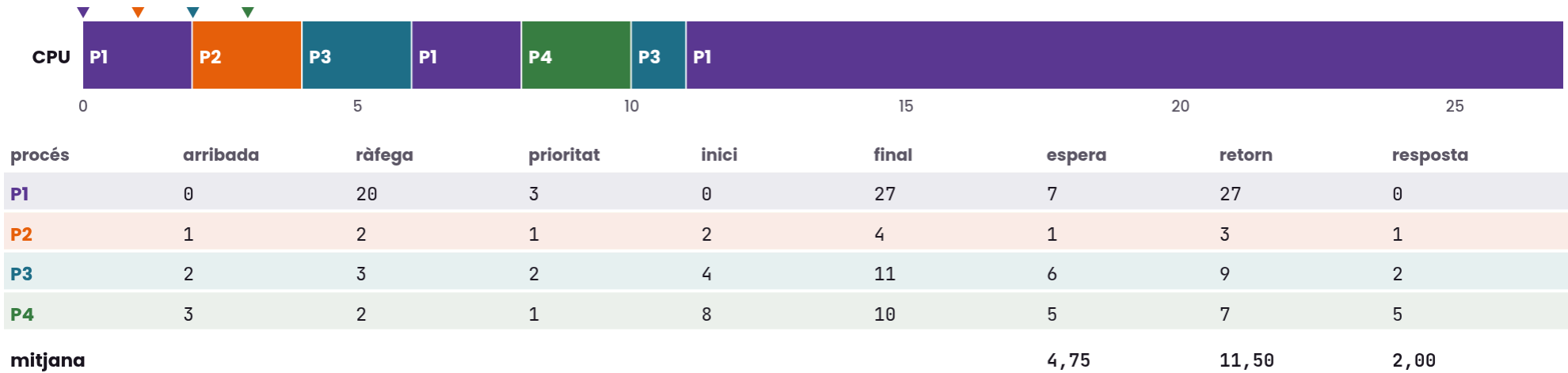
Estat, registres i comptador de programa, prioritat, temps de CPU (usuari/sistema), descriptors oberts, límits (`getrlimit`), senyals, taula de pàgines, fils

A Linux

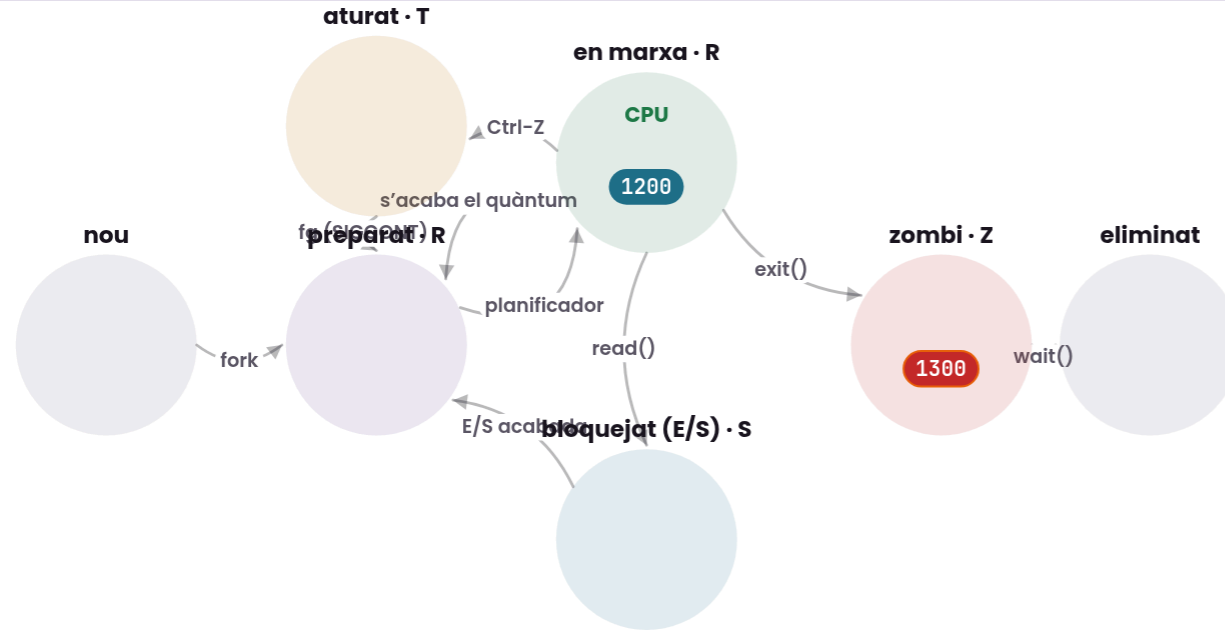
El PCB és `struct task_struct` (diversos KiB). S'inspecciona a `/proc/PID/`: `status`, `maps`, `fd/`, `limits` ... Així funcionen `ps` i `top`.

Canvi de context i planificació

Round Robin · quantum 2



Estats d'un procés



```
$ ps -o pid,ppid,stat,comm
  PID   PPID  STAT COMMAND
  1200   1100  R+   bash
  1300   1200  Z    lifecycle <defunct>
```

esdeveniments

- bash (1200) fa fork → neix 1300
- l'E/S de 1200 ha acabat (interrupció) → preparat
- 1300 acaba amb l'estat 7: queda zombi fins que el pare crida wait

Com es crea un procés?

Windows

`CreateProcess()` crea el procés i carrega el programa en un sol pas, amb moltes opcions.

Unix

1. `fork()` : un clon del procés actual
2. El clon ajusta el que vulgui (descriptors, directori...)
3. `exec()` : es converteix en un altre programa

Quatre famílies: **crear** (`fork`), **executar** (`exec...`), **acabar i senyalar** (`exit`, `kill` → U6), **esperar** (`wait...`).

Avui també: `posix_spawn()` (fork + exec en una crida), `vfork()` i `clone()` / `clone3()`, la crida real darrere de `fork` i dels fils (A4.7).

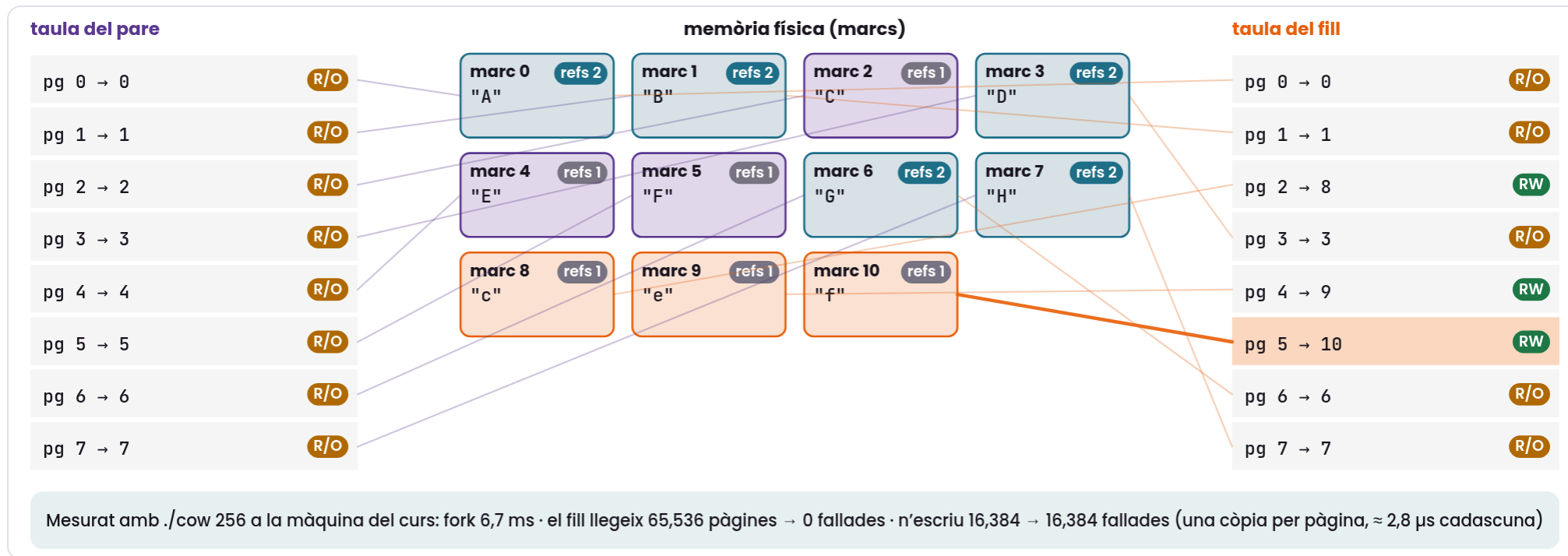
Clonar un procés: `fork` i el seu cost

- El fill executa el mateix programa amb una **còpia** de l'espai d'adreces del pare
- Crear el PCB és barat; copiar totes les pàgines seria molt car
- **Còpia en escriptura**: es comparteixen i només es copien en escriure

Sortida real

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096
bytes)
fork took 6.70 ms (only the page tables are
copied)
child reads 65536 pages: 0 page faults
(shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults
(one copy per page), 2753 ns per copy
parent: the child exited with status 0; my
copy still says 'x'
```

Còpia en escriptura, pàgina a pàgina



pid_t fork(void)

- Retorna **0 al fill**, el **PID del fill al pare** i -1 si falla
- Idèntics excepte el PID i el PPID (`getpid` , `getppid`)
- El fill hereta els gestors i la màscara de senyals, **no** els senyals pendents
- També hereta... la memòria intermèdia de stdio pendent

C tds/td3/fork_pid.c

```
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    printf("child %d, parent %d\n", getpid(), getppid());
    exit(EXIT_SUCCESS);
}

/* Parent: prints nothing after fork, but must wait for its child. */
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
```

Sortida real

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```


Memòries intermèdies de stdio i fork

```
printf("My pid is %d / ", getpid());
pid = fork();
/* child */ printf("child %d, parent %d\n", ...); exit(0);
/* parent */ waitpid(pid, ...); return 0;
```

el procés pare surt amb `exit()`: buida la seva memòria intermèdia

pare · stdout: completa

acabat

(buit)

fill · stdout: completa

acabat

(buit)

stderr: sense memòria intermèdia

crides al sistema

```
[hijo] write(1, "My pid is 356446 / child 356448, parent 356446\n", 47)
write(1, "My pid is 356446 / ", 19)
```

canonada → cat → terminal

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

sortida real (run.sh):

```
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Executar un altre programa: `execve`

C

```
int execve(const char *path, char *const argv[], char *const envp[]);  
/* execl, execlp, execl, execv, execvp, execvpe:  
   l = list, v = vector, p = search PATH, e = environment */
```

- `argv`: els arguments de `main`; `envp`: l'entorn (`PATH`, `HOME`, `LANG`, `LD_LIBRARY_PATH` ...; `getenv` / `setenv`)

- Si funciona, **no torna**: substitueix codi, dades, heap i pila
- Conserva el PID, el PPID i els descriptors oberts (excepte els `O_CLOEXEC`)
- Amb el bit SUID, l'UID efectiu passa a ser el del propietari del fitxer (`passwd`, `sudo`)
- Si falla, retorna -1

fork + dup2 + exec + wait : el shell treballant

launch · PID 356524

en marxa

ls -l demo · PID 356526

recollit per wait

descriptors del fill

```
0 → terminal
1 → out.txt
2 → terminal
3 → -
```

strace -f (traça real, resumida)

```
356524 execve("./launch", [ "./launch", "out.txt", "ls", "-l", "demo"], ...) = 0
356524 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
356524 write(3, "ls", ...) ...
356524 clone(..., flags=...|SIGCHLD) = 356526
356524 close(3) = 0
356524 wait4(356526, <unfinished ...>
356526 dup2(3, 1) = 1
356526 close(3) = 0
356526 execve("/usr/bin/ls", ...) = 0
356526 ... 19 crides de ls (openat, getdents64, close...) ...
356526 write(1, "a.txt\nb.txt\n", 12) = 12
356526 exit_group(0) = ?
356524 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) = 0}], 0, NULL) = 356526
356524 --- SIGCHLD {si_signo=SIGCHLD, ...} ---
356524 write(2, "launch: ls exited with status 0\n", ...)
```

paraula d'estat de wait

0x0000

```
WIFEXITED(s) = true
WEXITSTATUS(s) = 0
WIFSIGNALED(s) = false
```

Si cada procés neix d'un `fork`, qui és el primer?

Procés 0

Un per CPU (*idle*), construït pel nucli, en mode nucli: inicialitza la memòria virtual, crea els fils del nucli (`kthreadd`, PID 2) i el procés 1.

Procés 1

Passa a mode usuari i executa `/sbin/init`, que avui és **systemd**: inicia els serveis i adopta els orfes.

Sortida real

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
...
```

Acabar un procés: `_exit` i `exit`

C

```
void _exit(int status);    /* system call */  
void exit(int status);    /* atexit + flush stdio + _exit */
```

L'estat (0–255) el recull el pare amb `wait`; el shell el mostra a `$?`. Diferent de zero = error. Cap dels dos no torna.

- Es tanquen els fitxers i s'alliberen les pàgines (excepte les compartides)
- Els fills vius passen a `init` o al *subreaper* més proper
- El pare rep `SIGCHLD` (ignorat per defecte)
- Si era líder de sessió amb terminal, els altres reben `SIGHUP`
- Preferiu `exit`; feu servir `_exit` al fill després d'un `exec` fallit

Sessions i serveis (dimonis)

- Una **sessió** agrupa processos (normalment, els d'un terminal) amb un líder
- `setsid()`: crea una sessió nova sense terminal de control; -1 si ja és líder
- Dins de la sessió, **grups de processos** (una tasca del shell: `a | b`): `setpgid` / `getpgid`; `getsid` i `tcgetsid` consulten la sessió d'un procés i d'un terminal
- Dimoni clàssic: `fork`, el pare surt, el fill crida `setsid`

Actualització 2026

A Linux els serveis els llança **systemd**: el programa s'executa en primer pla i systemd gestiona la sessió, els registres (`journalctl`), els reinicis i els límits (cgroups). Ja no cal «dimonitzar-se».

Esperar un fill: `wait` i `waitpid`

C

```
pid_t wait(int *status);           /* = waitpid(-1, status, 0) */
pid_t waitpid(pid_t pid, int *status, int options);
/* pid > 0: that child · -1: any child · 0 / < -1: a process group
   options: WNOHANG (do not block) · WUNTRACED (stopped by SIGSTOP,
   SIGTSTP, SIGTTIN, SIGTTOU) · WCONTINUED (resumed by SIGCONT) */
```

Un fill acabat queda com a **zombi** fins al `wait`:

Sortida real

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has
called wait yet:
      PID    PPID STAT COMMAND
      356667 356666 Z    lifecycle
      waitpid → WIFEXITED = 1, WEXITSTATUS = 7;
now ps finds nothing:
      PID    PPID STAT COMMAND
...
```

Com ha acabat? Les macros W...

WIFEXITED → WEXITSTATUS	acabat amb <code>exit</code>
WIFSIGNALED → WTERMSIG	mort per un senyal
WIFSTOPPED → WSTOPSIG	aturat (amb <code>WUNTRACED</code>)
WIFCONTINUED	reprès (amb <code>WCONTINUED</code>)

C tds/td3/launch.c

```

int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
if (WIFEXITED(status)) {
    fprintf(stderr, "launch: %s exited with status %d\n",
            return WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s",
            signame(WTERMSIG(status)), WCOREDUMP(status)
    return 128 + WTERMSIG(status);
}

```


TD3 resol't + repte IA

- Les 5 variants de `buffer` en un terminal i en una canonada
- `fork_pid` i les seves tres correccions; `launch` amb tres finals
- Nou: `zones`, `lazy`, `cow` i `lifecycle` (mesures)

Bash

```
make -C tds && tds/tests/run.sh td3
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Per què fer fork d'un procés de 256 MiB triga mil·lisegons?
2. Què conserva un procés després d'`execve`?
3. Què és un zombi i qui l'elimina?